Máy tìm kiếm cho hust.edu.vn

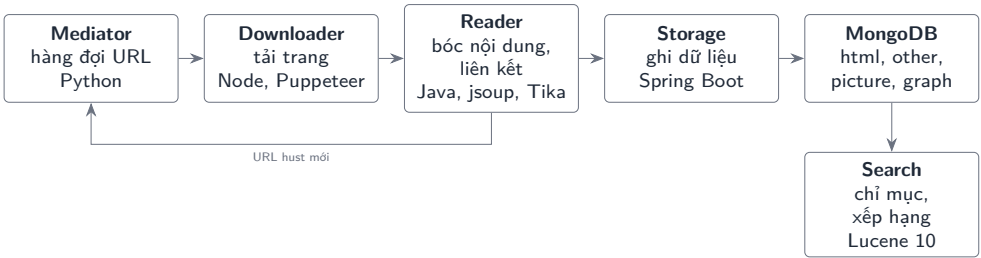
Kỹ thuật thu thập dữ liệu và tìm kiếm

Đồ án môn Tìm kiếm thông tin (IT4863)

Năm 2026

Kiến trúc tổng thể

Mục tiêu. Thu thập trang web, tài liệu và ảnh thuộc `*.hust.edu.vn`, sau đó cho phép tìm kiếm trên dữ liệu đã thu thập.



- Các dịch vụ độc lập, giao tiếp bằng HTTP và JSON, chạy bằng Docker Compose.
- Vòng lặp crawl: reader trả URL mới về mediator, nên tập trang được thu thập tự mở rộng theo liên kết.
- Search đọc dữ liệu đã lưu trong MongoDB và dựng chỉ mục riêng, tách khỏi quá trình crawl.

Crawl: nguồn URL ban đầu và cách mở rộng

Mục tiêu. Bắt đầu từ một tập URL mầm, sau đó tự tìm thêm trang mới qua liên kết.

1. **URL mầm.** File `link_all.txt` gồm khoảng 100 nghìn URL thuộc hust.edu.vn (trang chủ các đơn vị và trang con). Script `feed.py` gửi theo lô (mặc định 5000 URL) vào mediator qua `POST /add`. Nạp lại nhiều lần không gây trùng.
2. **Mở rộng theo liên kết.** Với mỗi trang đã tải, reader lấy mọi thẻ `<a href>`, giữ các URL thuộc hust.edu.vn và gửi lại mediator. URL chưa có trong hàng đợi được thêm vào và tải ở các lượt sau.
3. **Phạm vi.** Chỉ thu thập miền hust.edu.vn; liên kết ra ngoài được ghi vào đồ thị nhưng không tải.

- Hệ thống không đọc `sitemap.xml`: phạm vi crawl do danh sách mầm và đồ thị liên kết quyết định.
- Ví dụ: tải `https://hust.edu.vn/` cho ra các liên kết tới trang tin tức, trang đơn vị; các liên kết này vào hàng đợi rồi lần lượt được tải.

Crawl: hàng đợi URL và lập lịch

Mục tiêu. Quyết định URL nào được tải và lúc nào, bảo đảm không tải trùng và không gửi dồn dập vào một website.

1. Chuẩn hoá URL

- Chữ thường scheme và host, bỏ cổng mặc định, bỏ `..`, bỏ `#...`, bỏ tham số theo dõi (`utm_*`, `fbclid`), sắp xếp tham số. Ví dụ ở cuối slide.

2. Hàng đợi trên SQLite

- Bảng `urls`: `url`, `host`, `kind`, `next_due_at`, `attempts`. Bảng `hosts`: thời điểm gửi gần nhất của mỗi host.
- URL đã có thì giữ nguyên lịch cũ, nên gửi lại không gây tải trùng. Hàng đợi nằm trên đĩa, tắt máy không mất.

3. Chọn URL để giao

- Điều kiện: đã đến hạn (`next_due_at` $\leq$ hiện tại) và host đã nghỉ đủ 2 giây kể từ lần gửi trước (crawl lịch sự).
- Thứ tự: tài liệu trước, sau đó URL có hạn sớm nhất.

4. Giao cho downloader

- 4 bản downloader, chia lần lượt theo vòng.
- Hỏi `GET /ping`: rảnh mới gửi `POST /load`. Mỗi downloader tải một URL tại một thời điểm.
- Không trả lời trong 2 giây hoặc lỗi: tạm ngừng hỏi downloader đó 30 giây.

Ví dụ chuẩn hoá: `HTTPS://Hust.edu.vn:443/a/../b?z=2&a=1&utm_source=fb#top` $\rightarrow$ `https://hust.edu.vn/b?a=1&z=2`

Crawl: tải trang và bóc nội dung

Downloader

- Trang web: trình duyệt Chromium headless (Puppeteer), chờ sự kiện `load` rồi chờ mạng yên tối đa 2 giây để nội dung tải bằng JavaScript hiện ra.
- Tài liệu và ảnh (pdf, docx, jpg...): tải trực tiếp bằng `fetch`, mã hoá base64.
- Giới hạn 30 giây mỗi trang, 50 MB mỗi tệp. Lỗi chỉ ghi log, không gửi tiếp.

Reader với tài liệu

- Apache Tika bóc chữ của pdf, docx, xlsx. Ảnh không OCR, chỉ lấy tiêu đề và metadata.

Reader với trang web: tìm khối nội dung chính

- Bỏ `script`, `style`, `iframe`, phần tử ẩn.
- Mỗi đoạn chữ có ít nhất 25 ký tự (không tính thẻ a) cộng số ký tự của nó cho khối cha, và một nửa cho khối ông.
- Khối có tổng điểm cao nhất là nội dung chính; menu, chân trang có ít chữ liền mạch nên bị loại.

Tiêu đề và liên kết

- Tiêu đề: `og:title`, rồi `h1`, rồi thẻ `title`.
- Liên kết ra lấy từ toàn trang; văn bản liên kết là câu chứa thẻ a.

- Storage ghi bản ghi vào `html`, `other` hoặc `picture`, và mỗi liên kết thành một cạnh trong `graph`.

Crawl: cập nhật trang mới và trang đã thay đổi

Mục tiêu. Phát hiện trang mới đăng và cập nhật nội dung đã thay đổi mà không phải crawl lại toàn bộ.

1. Kiểm tra lại định kỳ theo loại URL

Loại	Nhận biết	Tải lại sau
Trang chủ (hub)	Đường dẫn rỗng hoặc `/`	1 ngày
Trang thường	Còn lại	7 ngày
Tài liệu, ảnh	Đuôi `.pdf`, `.docx`, `.jpg`, ...	30 ngày

- **Trang mới:** trang chủ được tải lại mỗi ngày. Bài mới đăng xuất hiện dưới dạng liên kết trên trang chủ, reader gửi URL đó về mediator và nó được tải ngay ở lượt sau.
- **Trang thay đổi:** khi tải lại, storage ghi đè nội dung và cập nhật `updatedAt`, giữ nguyên `createdAt`. Cạnh liên kết mới được thêm vào `graph`.
- **URL lỗi:** thử lại ngay ở lượt sau; lỗi 5 lần liên tiếp thì lùi một chu kỳ.
- **Chỉ mục:** search không cập nhật từng phần; gọi `POST /reindex` để dựng lại chỉ mục từ dữ liệu mới.
- Hệ thống không so sánh nội dung trước khi tải (ETag, Last-Modified): mỗi lần kiểm tra lại đều tải toàn bộ trang.

Search: tổng quan

Mục tiêu. Trả về các trang liên quan nhất cho một truy vấn tiếng Việt, có dấu hoặc không dấu.



- Hai giai đoạn: BM25 nhanh, chạy trên toàn kho; các tín hiệu tốn kém hơn chỉ tính lại cho 100 kết quả đầu.
- Trang rác và trang trùng vẫn nằm trong chỉ mục, chỉ bị loại khi truy vấn. Nhờ vậy có thể bật hoặc tắt từng kỹ thuật trên cùng một chỉ mục.

Search: phân tích văn bản tiếng Việt

Mục tiêu. Khớp được từ ghép tiếng Việt và truy vấn gõ không dấu mà không cần thư viện tách từ.

- Chuẩn hoá Unicode NFC cho mọi văn bản, vì cùng một chữ có dấu có hai cách mã hoá.
- Mỗi âm tiết là một token. Từ ghép được biểu diễn bằng bigram (cặp hai âm tiết liền nhau).
- Mỗi trường được lưu thêm một bản bỏ dấu.

Analyzer	Xử lý	Ví dụ: Bách Khoa Hà Nội
`vi`	tách âm tiết, chữ thường	`bách`, `khoa`, `hà`, `nội`
`viBi`	ghép bigram	`bách khoa`, `khoa hà`, `hà nội`
`viAscii`	bỏ dấu, đ thành d	`bach`, `khoa`, `ha`, `noi`
`urlTokens`	tách url theo ký tự không phải chữ, số	`.../hoc-phi-654580.html` thành `hoc`, `phi`, `654580`, `html`

- Truy vấn học phí khớp hai unigram `học`, `phí` và bigram `học phí`; trang chứa đúng cụm học phí được nhiều điểm hơn trang chỉ chứa rời hai âm tiết.

Search: các trường của chỉ mục và hàm điểm BM25

Mục tiêu. Mỗi phần của tài liệu được lưu thành trường riêng để chấm điểm với trọng số khác nhau.

Trường	Analyzer	Dùng để
`title`, `content`	`vi`	Khớp từng âm tiết. `content` lưu thêm vị trí từ cho truy vấn cụm và đoạn trích
`title_bi`, `content_bi`	`viBi`	Khớp cụm hai âm tiết
`title_ascii`, `content_ascii`	`viAscii`	Khớp truy vấn không dấu
`url_tokens`	`urlTokens`	Khớp từ trong url
`url`, `host`, `collection`	nguyên văn	Định danh, lọc theo website và loại tài liệu
`sig:pagerank`, `sig:recency`	số	Tín hiệu ngoài văn bản

BM25, tính riêng trên từng trường:

$$\mathrm{BM25}(q,d) = \sum_{t \in q} \mathrm{IDF}(t)\, \frac{f(t,d)\,(k_1+1)}{f(t,d) + k_1\left(1 - b + b\,\frac{|d|}{\mathrm{avgdl}}\right)}$$

- $f(t,d)$: số lần từ t xuất hiện trong trường; $|d|$: độ dài trường; avgdl: độ dài trung bình.
- $k_1 = 1.2$. $b = 0.75$ cho tiêu đề và url; $b = 0.5$ cho nội dung, để tài liệu dài như PDF không bị phạt độ dài quá nặng.

Search giai đoạn 1: truy hồi bằng BM25 nhiều trường

Mục tiêu. Nhanh chóng lấy 100 tài liệu có khả năng liên quan nhất từ toàn kho.

Trường	Trọng số
`title`	3
`title_bi`	3
`title_ascii`	2
`url_tokens`	1.5
`content_bi`	1.5
`content`	1
`content_ascii`	0.7

Cách tính

- Điểm của tài liệu là tổng điểm BM25 trên các trường nhân trọng số. Tài liệu chỉ cần khớp một token là được xét.
- Tiêu đề nặng gấp 3 nội dung; bigram nặng hơn âm tiết rời; bản bỏ dấu nhẹ hơn bản có dấu.

Truy vấn không dấu

- Nếu truy vấn không có dấu, ví dụ `lich thi`, chỉ dùng các trường bỏ dấu và url, để trang Lịch thi và trang chỉ có tên file không dấu được xét ngang nhau.

Bộ lọc

- Lọc theo loại tài liệu và theo website (ví dụ chỉ soict.hust.edu.vn), không ảnh hưởng đến điểm.

Search giai đoạn 2: chấm lại 100 kết quả đầu

Mục tiêu. Sắp xếp lại 100 kết quả bằng các tín hiệu chính xác hơn nhưng tốn kém hơn BM25.

Điểm mới = điểm BM25 + điểm của các tín hiệu sau:

Tín hiệu	Trọng số	Ý nghĩa
Cụm từ trong tiêu đề	2	Các từ của truy vấn đứng gần nhau đúng thứ tự, cho phép chèn tối đa 2 từ
Cụm từ trong nội dung	1	Như trên
PageRank	2	Trang được nhiều trang khác trỏ tới
Độ mới (khi `fresh=1`)	2	Trang mới cập nhật

Chuẩn hoá tín hiệu bằng hàm bão hoà

$$w \cdot \frac{s}{s + \text{pivot}}, \qquad s_{\text{PageRank}} = \max(\text{PR} \times N,\, 10^{-4}), \qquad s_{\text{độ mới}} = \frac{1}{1 + \text{số ngày từ lần cập nhật}/30}$$

- Hàm tăng nhanh rồi phẳng dần: một trang có PageRank rất lớn vẫn không lấn át được điểm văn bản.
- $\text{PR} \times N$ đưa giá trị trung bình về 1, không phụ thuộc số trang. Độ mới bằng 0.5 sau 30 ngày.

Search: PageRank trên đồ thị liên kết

Mục tiêu. Đo mức quan trọng của trang dựa trên cấu trúc liên kết: trang được nhiều trang quan trọng trỏ tới thì quan trọng.

Mô hình. Người duyệt web ngẫu nhiên: với xác suất 0.85 bấm một liên kết trên trang hiện tại, với xác suất 0.15 nhảy tới một trang bất kỳ.

$$\mathrm{PR}(v) = \frac{0.15}{N} + 0.85 \left(\sum_{u \to v} \frac{\mathrm{PR}(u)}{\mathrm{outdeg}(u)} + \frac{\mathrm{dangling}}{N} \right)$$

- $\mathrm{dangling}$: tổng PageRank của các trang không có liên kết ra, chia đều cho mọi trang để tổng luôn bằng 1.
- Dựng đồ thị từ `graph`: mỗi url là một nút, bỏ cạnh tự trỏ và cạnh trùng.
- Lặp từ giá trị khởi tạo $1/N$, tối đa 50 vòng, dừng khi tổng thay đổi giữa hai vòng nhỏ hơn 10^{-6}. Chi phí $O(\text{số vòng} \times (N + E))$.
- PageRank dùng ở hai chỗ: tín hiệu chấm lại, và chọn bản giữ lại trong nhóm trang trùng.

Search: phát hiện trang rác bằng độ lạ của từ vựng

Mục tiêu. Loại các trang bị chèn nội dung rác (từ khoá quảng cáo) khỏi kết quả.

Nhận xét. Trang thật của hust dùng từ vựng chung (đào tạo, sinh viên, thông báo). Trang bị chèn rác dùng nhiều từ hầu như không xuất hiện ở trang khác.

Độ lạ của trang p là IDF trung bình của các từ trong trang:

$$\text{độ lạ}(p) = \frac{1}{|W(p)|} \sum_{w \in W(p)} \log \frac{N}{\mathrm{df}(w)}$$

- N: số trang html có ít nhất 50 từ; $\mathrm{df}(w)$: số trang chứa từ w.
- Từ phổ biến cho giá trị gần 0; từ chỉ có ở một trang cho giá trị lớn nhất $\log N$.

Chọn ngưỡng tự động

1. Lập histogram độ lạ (bin rộng 0.1). Trang thật tạo một cụm lớn, trang rác tạo một cụm nhỏ ở phía cao hơn.
2. Làm mượt histogram đến khi còn đúng hai đỉnh; ngưỡng là đáy thung lũng giữa hai đỉnh, đo trên histogram gốc (phương pháp Prewitt và Mendelsohn).
3. Không tìm được hai đỉnh rõ ràng thì không đánh dấu trang nào.

- Trang vượt ngưỡng được gắn nhãn `spam`. Không cần đặt ngưỡng tay, vì ngưỡng thay đổi theo dữ liệu.

Search: khử trang trùng nội dung

Mục tiêu. Tránh kết quả lặp do bản sao, bản in hoặc trang mirror có nội dung giống hệt.

- **Khoá nội dung:** SHA-1 của nội dung sau khi chuẩn hoá NFC, chữ thường, gộp khoảng trắng. Hai trang cùng khoá được coi là trùng.
- Nội dung dưới 200 ký tự không tính khoá, vì quá ngắn để kết luận là trùng.

Chọn một bản giữ lại trong mỗi nhóm trùng

1. Trang có PageRank cao nhất.
2. Hoà thì chọn url ngắn hơn.
3. Trang đã bị đánh dấu rác không được chọn.

- Các trang còn lại trong nhóm được gắn nhãn dup và bị loại khi truy vấn.
- Ví dụ: cùng một thông báo được đăng ở trang tin tức và trang đơn vị; chỉ trang có PageRank cao hơn xuất hiện trong kết quả.

Search: bốn cấu hình xếp hạng

Mục tiêu. Bật lần lượt từng kỹ thuật để đo đóng góp của mỗi kỹ thuật trên cùng một chỉ mục.

Profile	Truy hồi	Chấm lại	Loại rác, trùng
A	BM25 trên tiêu đề, nội dung, url	không	không
B	A + bigram + bỏ dấu	không	không
C	như B	cụm từ, PageRank	không
D (mặc định)	như B	như C	có

- Ví dụ truy vấn học phí với profile D: truy hồi trên 7 trường, loại trang rác và trùng, lấy 100 kết quả; trang có cụm học phí trong tiêu đề và trang có PageRank cao được cộng điểm; trả 10 kết quả đầu kèm đoạn trích có từ khoá được tô đậm.

Search: phương pháp đánh giá

Mục tiêu. So sánh các profile A đến D bằng nhãn liên quan do người gán.

- **Truy vấn:** 25 truy vấn gồm có dấu, không dấu và tên đơn vị, ví dụ `học phí 2025`, `lich thi`, `soict tuyển sinh`.
- **Nhãn:** 0 không liên quan, 1 liên quan một phần, 2 liên quan hoàn toàn.
- **Pooling:** gộp 10 kết quả đầu của cả 4 profile để gán nhãn, nên mọi trang được bất kỳ profile nào xếp cao đều được xem xét.

Độ đo trên 10 kết quả đầu

$$\mathrm{P@10} = \frac{\text{số kết quả có nhãn} \geq 1}{10}, \qquad \mathrm{nDCG@10} = \frac{1}{\mathrm{IDCG@10}} \sum_{i=1}^{10} \frac{2^{g_i} - 1}{\log_2(i+1)}$$

- P@10 đo tỷ lệ kết quả liên quan. nDCG@10 thưởng nhiều hơn khi trang liên quan hoàn toàn đứng ở vị trí cao.
- So sánh B với A cho thấy đóng góp của bigram và bỏ dấu; C với B: cụm từ và PageRank; D với C: loại rác và trùng.

Giới hạn và hướng mở rộng

- **Crawl:** chưa đọc `sitemap.xml`; chưa dùng ETag hoặc Last-Modified nên mỗi lần kiểm tra lại đều tải toàn bộ trang.
- **Chỉ mục:** dựng lại toàn bộ mỗi lần, chưa cập nhật từng phần; tín hiệu độ mới lệch dần nếu không dựng lại.
- **Tách từ:** dùng bigram thay cho tách từ tiếng Việt; hướng mở rộng là dùng pyvi.
- **PageRank:** mọi liên kết có trọng số như nhau, nên liên kết ở menu và chân trang làm tăng điểm trang chủ; chưa dùng anchor text.
- **Trang rác:** không phát hiện được nếu số trang rác quá ít để tạo cụm riêng trên histogram.
- **Tính năng:** chưa có gợi ý khi gõ và sửa chính tả.